



DATA-547 PROJECT REPORT

By: Grant Gonnerman, Miguel Magno



Abstract

This paper explores multiple Convolutional Neural Networks (CNNs) used to identify three types of lung cancer. More Americans die from lung cancer than any other type of cancer across both men and women. Getting an early diagnosis is critical to the full recovery of the patient. Using various model architectures in testing we found what was most effective to classify the types of lung cancer with the highest accuracy.

FALL 2024
GRAND VIEW UNIVERSITY

Table of Contents

Abstract.....	1
Introduction.....	3
Data Description and Exploratory Data Analysis	3
Modeling and Methodology	5
Conclusion.....	9
Areas of Further Study and Limitations	9
References	10

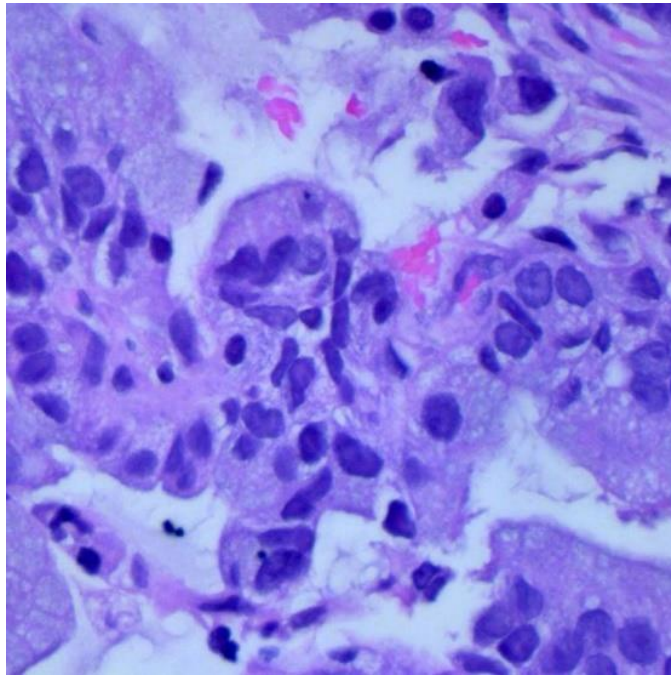
Introduction

Lung cancer is the third most common cancer in the United States and more Americans die from lung cancer than any other type of cancer across both men and women. When detected in an early stage, lung cancer is highly treatable but if it is misdiagnosed or not identified it may progress and therefore become harder to treat². It is common for early signs to be missed as a recent study found nearly 38% of all cancer-related medical malpractice claims were due to lung cancer misdiagnoses³. The goal for this project is to build a model to accurately detect types of lung cancer to assist doctors in their diagnosis.

Data Description and Exploratory Data Analysis

This dataset was found on Kaggle¹ but originally published by Andrew Borkowski M.D. at (<https://arxiv.org/abs/1912.12142v1>). The dataset contains 15,000 images of three types of lungs. Each class contains 5,000 images.

The three types of lungs included in this dataset are as follows:

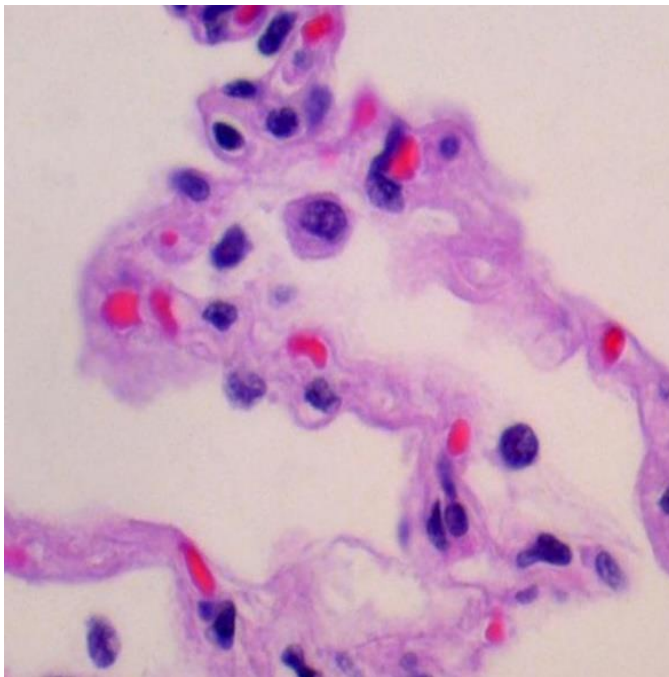
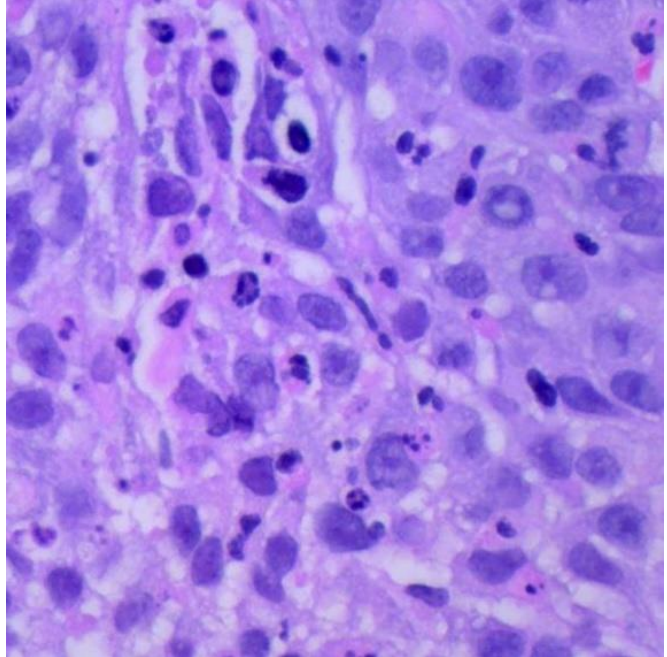


Adenocarcinoma

Adenocarcinoma is a type of non-small cell lung cancer (NSCLC) that starts in cells in the lung that make mucus, called epithelial cells. This form of lung cancer is the most common form of NSCLC⁴.

Squamous Cell Carcinoma

Squamous Cell Carcinoma is another kind of NSCLC. This form of cancer starts in squamous cells, which are flat cells that line the inside of the airways in the lungs. They are often linked to a history of smoking and tend to be found in the central part of the lungs, near a main airway⁴.



Benign

These images represent healthy lungs, serving as a control group to benchmark the model's ability to differentiate normal lung tissue from cancerous formations.

Modeling and Methodology

<https://www.kaggle.com/code/mjmagno/basic-cnn-for-cancer-images>

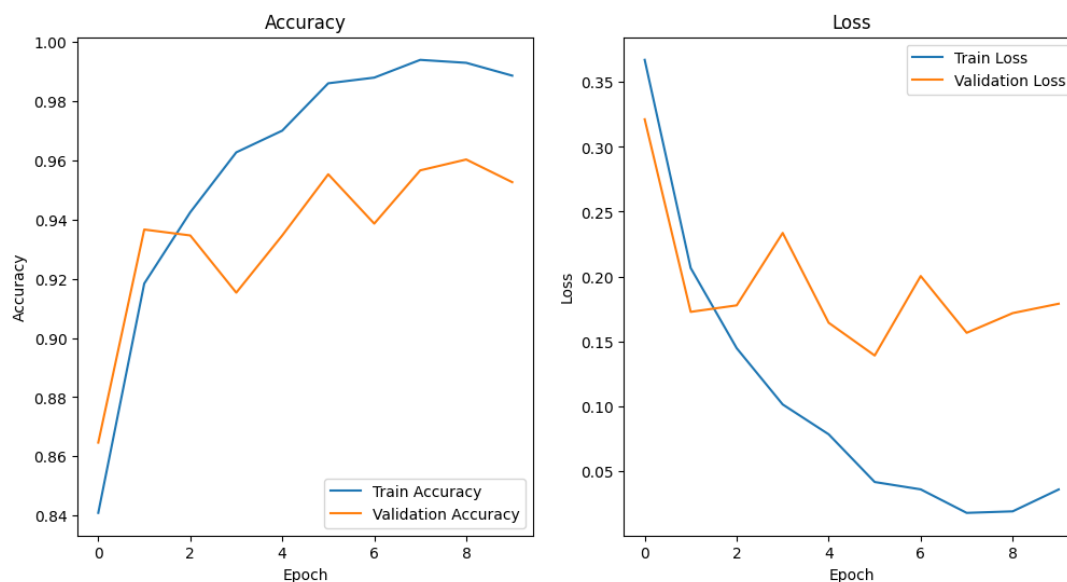
Model 1: Baseline Model

```
md1 = tf.keras.models.Sequential([
    tf.keras.layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16, kernel_size=(3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
    tf.keras.layers.Conv2D(32, kernel_size=(3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])
md1.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history = md1.fit(train_ds, validation_data = val_ds, epochs=10)
```

For our baseline model, we chose straightforward architecture. The model begins with a Rescaling layer to normalize the pixel values of the RGB images to the range [0, 1]. This is followed by three convolutional layers, each using ReLU activation for non-linearity, and each convolutional layer is followed by a max pooling layer to reduce spatial dimensions and prevent overfitting. After the convolutional layers, the model includes a fully connected (Dense) layer with ReLU activation for feature extraction. Finally, we compiled the model using the Adam optimizer and the sparse categorical crossentropy loss function, which is appropriate for multi-class classification with integer labels.

Training and Validation Results



From these visualizations, we can evaluate our model's performance on a single run, achieving a validation accuracy of 95.2%. This result indicates strong initial performance. Moving forward, we aim to further enhance this accuracy through iterative improvements and potential model refinement.

Model 2: Neurons and Layers testing

We evaluated many different architectures before arriving at our final model. First, we tested adding more and less neurons per layer, adding more and less layers, different dropout layers, activation functions, and optimizers. Ultimately, we found that more neurons than the baseline model decreases our accuracy score. We found success decreasing the number of neurons from 128 to 32 in the last dense layer increases accuracy but at a point we saw diminishing returns as we tested even lower values. Below is the layers and neuron architecture combination we saw the best results with.

```
md2 = tf.keras.models.Sequential([
    tf.keras.layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(32, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])
md2.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history = md2.fit(train_ds, validation_data = val_ds, epochs=10)
```

With this model we were able to achieve an average accuracy of 96.8%. This gave us a strong improvement and decided to build off this model with the remainder of our testing.

Model 3: Dropout layers

Next, we experimented with Dropout. Dropout layers through the entire network did not seem to work well. We saw a decrease in accuracy of about 2% but dropout layers either after the convolutional blocks or between the dense layers led to a slight bump in accuracy from our baseline model. Our best results were from using a dropout layer after the first dense layer.

```

md3 = tf.keras.models.Sequential([
    tf.keras.layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(32, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(3, activation='softmax')
])
md3.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history = md3.fit(train_ds, validation_data = val_ds, epochs=10)

```

With this model we were able to achieve an average accuracy of 95.8%. This score was a decrease in accuracy from model 2 but still a slight increase from the baseline.

Model 4: Optimizers

Finally, we assessed multiple optimizers. From our testing we saw varying results. With SGD we saw significantly worse performance. We did see slightly better performance with RMSProp, but we experienced extremely high fluctuations in our validation accuracy/loss scores. Of note, we also experimented with different learning rates for each optimizer but found that deviations from the default resulted in, at best, slightly worse accuracy and, at worst, significantly poorer outcomes.

```

md4 = tf.keras.models.Sequential([
    tf.keras.layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(32, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])
md4.compile(optimizer='RMSprop', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history = md4.fit(train_ds, validation_data = val_ds, epochs=10)

```

With this model we were able to achieve an average accuracy of 96.7%. This also made a strong improvement over the baseline model and was similar in performance to our best performing model, model 2.

What didn't work

We also tried various activation functions in both the convolutional layer and the dense layers of our model. We experimented with LeakyReLU, ELU, Swish and GELU. GELU and LeakyReLU performed the closest to our baseline, but all 4 activation functions worked decently well. Ultimately, we decided to stick with ReLU for our final model, but these 4 activation functions are worth exploring further in different combinations.

Final model

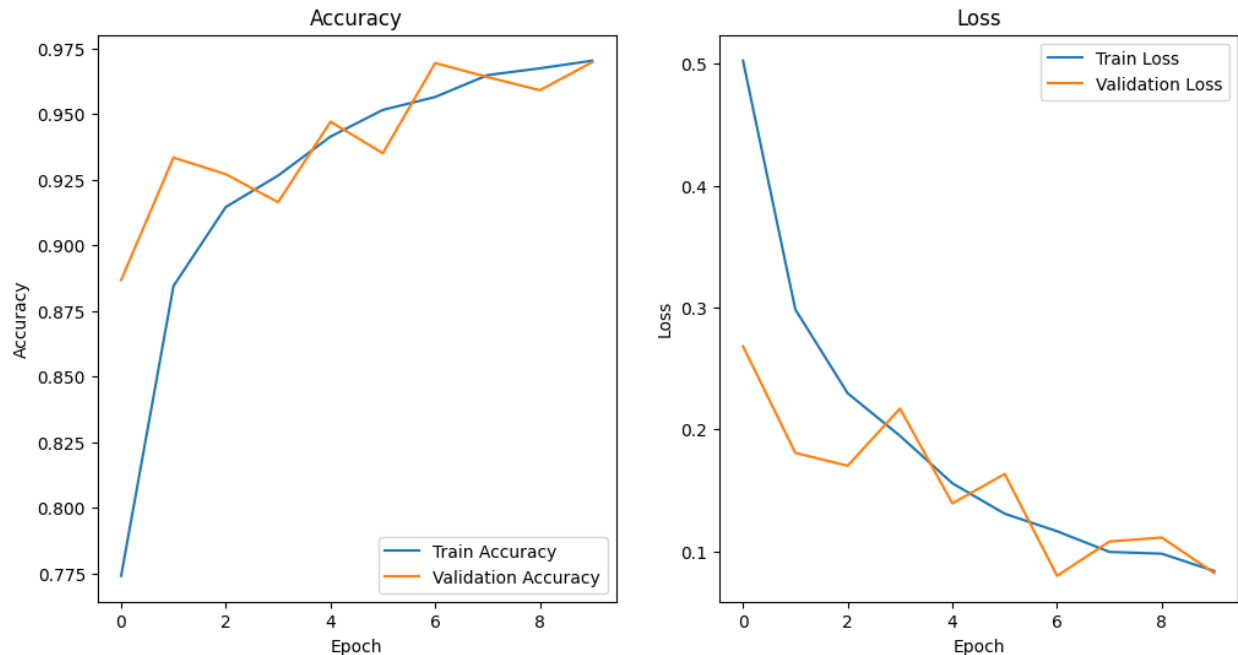
After many rounds of testing, we found increasing the number of neurons has a negative impact on our performance while decreasing the neurons in the final layer and adding another convolutional layer offered improvements in our accuracy score. We also found that adding a dropout layer after the first dense layer offered a slight boost in performance along with the RMSprop optimizer. To get our final model we compiled the best parts of each of the previous 4 models. The specifics of the model's architecture can be seen in the image below.

```
md5 = tf.keras.models.Sequential([
    tf.keras.layers.Rescaling(1./255, input_shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(32, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(3, activation='softmax')
])
md5.compile(optimizer='RMSprop', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history = md5.fit(train_ds, validation_data = val_ds, epochs=10)
```

With this architecture we were able to achieve a validation accuracy score of 96.9% which is an improvement of 1.7% over our baseline model.

Training and Validation Results



From these visualizations, we can evaluate our model's performance on a single run. These results indicate strong improvement in our accuracy score over our baseline model.

Conclusion

Lung cancer is the leading cause of cancer deaths in the US, accounting for about 1 in 5 of all cancer deaths. Early detection is vital for survival, and using neural networks, doctors can more easily identify cancerous lungs in earlier stages. Throughout this paper, we have explored how a deep Convolutional Neural Network (CNN) can be used to predict the type of lung cancer present in an image of a human lung, identifying either Adenocarcinoma or Squamous Cell Carcinoma.

After building a baseline model that achieved a validation accuracy of 95.2%, we experimented with a wide variety of hyperparameters. By trying different combinations of optimizers, activation functions, image resolutions, dropout layers, and deeper networks, we ultimately decided to use a model with four convolutional layers and one dense layer, one dropout layer and RMSprop for the optimizer. This model achieved an accuracy of 96.9%, an improvement of 1.7% over our baseline.

Areas of Further Study and Limitations

Although we achieved an accuracy of 96.9%, there is still significant room for improvement in any deep neural network trained on this dataset. Our two biggest limitations in creating

an optimal neural network were computational resources and time. These restrictions caused several challenges throughout the process. We were unable to increase the number of epochs the model could train for and were also unable to validate our results through multiple-fold testing. Additionally, due to the aforementioned constraints, we were unable to try as many hyperparameter combinations as we would have liked.

If we were to continue working on this project, we would like to further investigate how different optimizers interact with deeper networks and explore how various activation functions, in combination with different dropout configurations, could improve the accuracy of our model.

References

Data:

<https://www.kaggle.com/datasets/rm1000/lung-cancer-histopathological-images/data>¹

Lung cancer background information:

<https://www.cdc.gov/lung-cancer/statistics/index.html>²

<https://www.verywellhealth.com/risks-of-misdiagnosed-lung-cancer-and-delayed-treatment-5248414>³

<https://www.cancer.org/cancer/types/lung-cancer/about/what-is.html>⁴